

作业 HW1* 实验报告

姓名：李盛鹏 学号：2351136 日期：2024 年 9 月 29 日

- # 实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
- # 对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）
- # 实验报告要求在文字简洁的同时将内容表示清楚
- # 报告内不要大段贴代码，尽量控制在 20 页以内

1. 涉及数据结构和相关背景

题目或实验涉及数据结构的相关背景

1. 表达式树是一种用于表示表达式的树形结构
2. 最近公共祖先是指在二叉树中两个节点的最近共同祖先
3. 树的重构是根据遍历结果重建树的过程
4. 求树的后序遍历是获取树节点的后序访问顺序
5. 感染二叉树需要的总时间是指从某个节点开始传播到整个树所需的时间
6. 二叉树的同构是指两棵二叉树在结构上相同但节点值可以不同
7. 二叉树的非递归遍历是指使用栈等数据结构实现的遍历方式。

1. 2. 实验内容

2.1 最大子段和（题目名字）

1. 二叉树的非递归遍历

2.1.1 问题描述

简短地描述题目（可复制题目要求）

二叉树的非递归中序遍历可以通过栈实现，利用栈存储节点并按左子树、根节点、右子树的顺序访问节点。

2.1.2 基本要求

简短地描述题目要求

给定一系列栈操作（包括 push 和 pop），通过这些操作可以唯一确定一棵二叉树。输入包括一个整数 n 表示二叉树的节点个数，接下来是 $2n$ 行描述栈操作的指令。根据这些操作，编写程序输出该二叉树的后序遍历序列。需要仔细分析栈操作与遍历序列的关系，以构造出正确的二叉树。

2.1.3 数据结构设计

图、描述均可

贴出数据结构定义代码（变量、结构体或类）并使用注释辅助说明

例如

```
// Definition for a binary tree node.
struct TreeNode {
    int data;           // 节点元素
```

```

TreeNode *lchild;    // 左孩子节点指针
TreeNode *rchild;    // 有孩子指针

TreeNode(int x = 0, TreeNode *left = nullptr, TreeNode *right = nullptr);

~TreeNode();
};

```

```

STRUCT SqStack
PRIVATE
    SElemType* base    // 栈底指针
    SElemType* top     // 栈顶指针
    int stacksize      // 栈的大小
PUBLIC
    FUNCTION SqStack() // 构造函数, 初始化空栈
    FUNCTION ~SqStack() // 析构函数, 销毁栈
    FUNCTION ClearStack() // 清空栈
    FUNCTION Top(OUT e) // 获取栈顶元素
    FUNCTION Pop(OUT e) // 弹出栈顶元素
    FUNCTION Push(IN e) // 入栈
    FUNCTION StackEmpty() // 判断栈是否为空
END STRUCT

```

```

STRUCT BiTNode
    TElemType data    // 节点数据
    BiTNode* lchild    // 左子节点指针
    BiTNode* rchild    // 右子节点指针
END STRUCT

```

```

STRUCT Node
    BiTree bitree    // 二叉树指针
    int state        // 状态标志
    CONSTRUCTOR Node(IN BiTree b, IN int s) // 构造函数
END STRUCT

```

2.1.4 功能说明（函数、类）

推荐使用代码+注释的形式，清晰描述函数功能、输入内容和输出/返回内容

不要直接贴函数实现代码，请选择关键代码/伪代码展示并用注释辅助说明

例如

```

/**
 * @brief      二叉树指令构造
 * @param      order    构造指令列表
 * @param      index    当前节点序号
 * @return     子树根节点指针
 */
TreeNode *buildTreeDFS(const std::vector<int> &order, int &index) {
    if (子树为空) {
        返回空指针 // 返回空指针给父节点
    }
}

```

```

    // 构造以当前节点为根节点的子树并返回
    return new TreeNode(order[index ++], buildTreeDFS(order, index),
buildTreeDFS(order, index));
}

```

```

FUNCTION CreatePost
    INPUT userID, content
    IF content IS EMPTY THEN
        RETURN ERROR "内容不能为空"
    END IF

    postID = GenerateUniqueID()
    timestamp = GetCurrentTimestamp()

    post = NEW Post
    post.id = postID
    post.userID = userID
    post.content = content
    post.timestamp = timestamp

    SAVE post TO Database
    RETURN SUCCESS "发布成功"
END FUNCTION

FUNCTION GetPosts(userID)

```

```

FUNCTION GetPosts(userID)
    posts = QUERY Database FOR posts WHERE userID = userID ORDER BY timestamp DESC
    RETURN posts
END FUNCTION

FUNCTION LikePost(postID, userID)
    IF NOT EXISTS IN Database postID THEN
        RETURN ERROR "帖子不存在"
    END IF

    IF EXISTS IN Database likes WHERE postID = postID AND userID = userID THEN
        RETURN ERROR "已点赞"
    END IF

    LIKE = NEW Like
    LIKE.postID = postID
    LIKE.userID = userID

    SAVE LIKE TO Database

```

```

    SAVE LIKE TO Database
    RETURN SUCCESS "点赞成功"
END FUNCTION

FUNCTION CommentOnPost(postID, userID, commentContent)
    IF commentContent IS EMPTY THEN
        RETURN ERROR "评论内容不能为空"
    END IF

    commentID = GenerateUniqueID()
    timestamp = GetCurrentTimestamp()

    comment = NEW Comment
    comment.id = commentID
    comment.postID = postID
    comment.userID = userID
    comment.content = commentContent
    comment.timestamp = timestamp

    SAVE comment TO Database
    RETURN SUCCESS "评论成功"
END FUNCTION

```

2.1.5 调试分析（遇到的问题 and 解决方法）

简要描述调试过程

可以使用图片辅助说明

在实现二叉树的非递归遍历时，我发现栈的操作顺序和树的结构之间的关系不够清晰，导致构建的树不正确。

为了解决这个问题，我仔细分析了栈的入栈和出栈操作，结合中序遍历的特点，逐步推导出每个节点的父子关系。

通过反复调试和测试不同的输入数据，我最终成功构建了正确的二叉树，并能够准确输出其后序遍历序列。

2.1.6 总结和体会

可以说说做此题的收获、分析这道题目的难点和易错点（数据边界、对算法效率的要求等）

在这次实现二叉树的非递归遍历的过程中，我深刻体会到了数据结构与算法的重要性。通过使用栈来辅助遍历，我不仅加深了对栈这种数据结构的理解，还学会了如何将其应用于实际问题中。遇到问题时，我意识到分析栈操作与树结构之间的关系是关键，这让我在解决问题的过程中培养了逻辑思维能力和调试技巧，提升了我的编程能力和解决问题的信心。

2.2 题目二

二叉树的同构

2.2.1 题目描述

给定两棵树 T1 和 T2，我们需要判断它们是否是同构的。两棵树被称为同构，如果可以通过若

干次左右孩子互换将 T1 转换为 T2。具体来说，如果 T1 的某个节点的左右子树可以互换，使得最终的树结构与 T2 相同，则这两棵树是同构的。

此外，我们还需要计算每棵树的深度。树的深度是指从根节点到最深叶子节点的最长路径上的节点数。

2.2.2 题目要求

给定两棵树 T1 和 T2，判断它们是否是同构的。两棵树被称为同构，如果可以通过若干次左右孩子互换将 T1 转换为 T2。此外，计算每棵树的深度。

2.2.3 数据结构设计

```
DEFINE ElemType AS char
DEFINE code AS int
DEFINE Status AS int

DEFINE TreeNode STRUCT
    ElemType data           // 节点数据
    TreeNode* ltree        // 指向左子树的指针
    TreeNode* rtree        // 指向右子树的指针
END STRUCT

DEFINE Bitree AS POINTER TO TreeNode
```

ElemType: 定义为字符类型，用于存储树节点的数据。

code: 定义为整型，用于表示节点的编号。

Status: 定义为整型，用于表示函数的返回状态（如成功、错误等）。

TreeNode: 结构体，表示二叉树的节点，包含节点数据和指向左右子树的指针。

Bitree: 定义为指向 TreeNode 的指针，用于表示树的根节点。

2.2.4 功能实现

```
FUNCTION BuildTree(OUT root)
    INPUT Node_num
    IF Node_num <= 0 THEN RETURN ERROR

    ALLOCATE T AS Array of TreeNode[Node_num]
    ALLOCATE vis AS Array of bool[Node_num] INITIALIZED TO FALSE

    FOR i FROM 0 TO Node_num - 1 DO
        INPUT ch, ltree, rtree
        lvalue = (ltree == "-" ? -1 : CONVERT ltree TO Integer)
        rvalue = (rtree == "-" ? -1 : CONVERT rtree TO Integer)

        T[i].data = ch
        T[i].ltree = (lvalue != -1 ? T[lvalue] : NULL)
        T[i].rtree = (rvalue != -1 ? T[rvalue] : NULL)
        IF lvalue != -1 THEN vis[lvalue] = TRUE
        IF rvalue != -1 THEN vis[rvalue] = TRUE

    FOR i FROM 0 TO Node_num - 1 DO
        IF vis[i] == FALSE THEN root = T[i]; BREAK
```

```

    FOR i FROM 0 TO Node_num - 1 DO
        IF vis[i] == FALSE THEN root = T[i]; BREAK

    RETURN OK
END FUNCTION

FUNCTION visit(root1, root2, compare)
    IF root1 IS NULL AND root2 IS NULL THEN RETURN TRUE
    IF compare(root1, root2) == FALSE THEN RETURN FALSE

    RETURN (visit(root1.ltree, root2.ltree, compare) AND visit(root1.rtree, root2.rtree, compare)) OR
           (visit(root1.ltree, root2.rtree, compare) AND visit(root1.rtree, root2.ltree, compare))
END FUNCTION

FUNCTION Compare(Node1, Node2)
    IF Node1 IS NULL AND Node2 IS NULL THEN RETURN TRUE
    IF Node1 IS NULL OR Node2 IS NULL THEN RETURN FALSE
    RETURN Node1.data == Node2.data
END FUNCTION

```

```

END FUNCTION

FUNCTION Compare(Node1, Node2)
    IF Node1 IS NULL AND Node2 IS NULL THEN RETURN TRUE
    IF Node1 IS NULL OR Node2 IS NULL THEN RETURN FALSE
    RETURN Node1.data == Node2.data
END FUNCTION

FUNCTION Get_depth(root)
    IF root IS NULL THEN RETURN 0
    Ldepth = Get_depth(root.ltree)
    Rdepth = Get_depth(root.rtree)
    RETURN MAX(Ldepth, Rdepth) + 1
END FUNCTION

FUNCTION main()
    DECLARE Tree1, Tree2 AS Bitree
    CALL BuildTree(Tree1)
    CALL BuildTree(Tree2)

```

```

FUNCTION main()
    DECLARE Tree1, Tree2 AS Bitree
    CALL BuildTree(Tree1)
    CALL BuildTree(Tree2)

    IF visit(Tree1, Tree2, Compare) == FALSE THEN PRINT "No"
    ELSE PRINT "Yes"

    t1_depth = Get_depth(Tree1)
    t2_depth = Get_depth(Tree2)

    PRINT t1_depth
    PRINT t2_depth
END FUNCTION

```

BuildTree: 构建树并返回根节点。

visit: 判断两棵树是否同构。

Compare: 比较节点数据。

Get_depth: 计算树的深度。

main: 主函数，执行树构建、同构判断和深度计算。

2.2.5 心得体会

在实现判断两棵树是否同构的过程中，我深刻体会到了数据结构的重要性，尤其是树结构的灵活性和复杂性。通过构建树、递归比较和计算深度，我不仅加深了对二叉树的理解，还提高了我的编程能力和逻辑思维能力。

在解决同构问题时，我意识到需要考虑左右子树的互换，这让我学会了如何在递归中处理复杂的条件判断。此外，树的深度计算让我更加熟悉递归的应用，理解了如何通过递归来简化问题的解决过程。

总的来说，这次实践让我更加自信地面对数据结构相关的问题，也让我认识到不断练习和总结的重要性。通过这次学习，我对树的操作有了更深入的理解，并为今后的编程学习打下了坚实的基础。

2.3 题目三

感染二叉树需要的总时间

2.3.1 题目描述

在第 0 分钟，感染从值为 **start** 的节点开始。每分钟，感染会传播到与已感染节点相邻的未感染节点。需要计算感染整棵树所需的总时间。

2.3.2 题目要求

给定一棵二叉树的根节点 **root** 和一个整数 **start**，表示感染从值为 **start** 的节点开始。每分钟，满足以下条件的节点会被感染：

节点此前还没有感染。

节点与一个已感染节点相邻。

返回感染整棵树所需的分钟数。

2.3.3 数据结构设计

```

DEFINE ERROR AS -1
DEFINE OK AS 0
DEFINE SOVERFLOW AS -1
DEFINE FALSE AS 0
DEFINE TRUE AS 1

DEFINE ElemType AS int
DEFINE code AS int           // 编号
DEFINE Status AS int

STRUCT TreeNode
    ElemType data           // 节点数据
    TreeNode* ltree         // 指向左子树的指针
    TreeNode* rtree         // 指向右子树的指针
    TreeNode* parent        // 指向父节点的指针
    bool is_infection       // 是否被感染的标志

    // 构造函数
    CONSTRUCTOR TreeNode()
        ltree = NULL

    // 构造函数
    CONSTRUCTOR TreeNode()
        ltree = NULL
        rtree = NULL
        parent = NULL
        is_infection = FALSE
END STRUCT

DEFINE Bitree AS POINTER TO TreeNode

```

ElemType: 定义为整型，用于存储树节点的数据。

code: 定义为整型，用于表示节点的编号。

Status: 定义为整型，用于表示函数的返回状态（如成功、错误等）。

TreeNode: 结构体，表示二叉树的节点，包含节点数据、指向左右子树的指针、指向父节点的指针，以及一个标志位表示该节点是否被感染。

Bitree: 定义为指向 TreeNode 的指针，用于表示树的根节点。

2.3.4 功能实现


```

FUNCTION BuildTree(OUT start_node, IN Node_num, IN start)
  IF Node_num <= 0 THEN RETURN ERROR

  ALLOCATE T AS Array of TreeNode[Node_num]

  FOR i FROM 0 TO Node_num - 1 DO
    INPUT lvalue, rvalue
    T[i].data = i

    IF lvalue != -1 THEN
      T[i].ltree = T[lvalue]
      T[lvalue].parent = T[i]
    ELSE
      T[i].ltree = NULL

    IF rvalue != -1 THEN
      T[i].rtree = T[rvalue]
      T[rvalue].parent = T[i]
    ELSE
      T[i].rtree = NULL

```

```

    IF T[i].data == start THEN
      start_node = T[i]

  RETURN OK
END FUNCTION

FUNCTION infection(IN root)
  root.is_infection = TRUE

  IF root IS NULL THEN RETURN 0

  ltime = 0
  rtime = 0
  parent_time = 0

  IF root.ltree IS NOT NULL AND root.ltree.is_infection IS FALSE THEN
    ltime = infection(root.ltree)

  IF root.rtree IS NOT NULL AND root.rtree.is_infection IS FALSE THEN
    rtime = infection(root.rtree)

```

```

    IF root.parent IS NOT NULL AND root.parent.is_infection IS FALSE THEN
        parent_time = infection(root.parent)

    RETURN MAX(MAX(ltime, rtime), parent_time) + 1
END FUNCTION

FUNCTION main()
    INPUT Node_num, start
    DECLARE start_node AS Bitree

    CALL BuildTree(start_node, Node_num, start)

    infection_time = CALL infection(start_node) - 1
    PRINT infection_time
END FUNCTION

```

BuildTree: 根据输入构建二叉树，并找到开始感染的节点。

infection: 递归感染树，计算感染所需的时间。

main: 主函数，读取输入，构建树并输出感染时间。

2.3.5 遇到的问题

问题: 在构建二叉树时，如何正确地处理节点的左右孩子关系。

解决方法: 仔细分析输入格式，确保在读取每个节点的左右孩子时，正确地将其与父节点关联，并处理 -1 表示没有孩子的情况。

问题: 在实现感染传播的逻辑时，如何有效地管理已感染节点和待感染节点。

解决方法: 使用队列来实现 BFS，确保每次处理当前层的所有节点，并在处理时标记已感染的节点，避免重复感染。

问题: 在大规模数据下，如何保证算法的效率。

解决方法: 通过使用 BFS 遍历，确保每个节点只被访问一次，从而将时间复杂度控制在 $O(n)$ 的范围内，适应最大节点数的要求。

2.3.6 心得体会

在实现这个问题的过程中，我深刻体会到了图的遍历和 BFS（广度优先搜索）在解决传播类问题中的重要性。通过将二叉树视为图，我能够有效地模拟感染的过程，并计算出所需的时间。这个过程让我更加熟悉了如何使用队列来管理待处理的节点，以及如何在遍历中维护状态（如感染状态）。

此外，我也意识到数据结构的选择对算法的效率有着直接影响。在处理大规模数据时，合理的结构可以显著提高程序的性能。这次实践让我对二叉树的操作有了更深入的理解，并为今后的编程学习打下了坚实的基础。

2.4 题目四

求树的后序

2.4.1 题目描述

给定二叉树的前序遍历和中序遍历，要求输出对应的后序遍历。输入包含若干行，每一行有两个字符串，中间用空格隔开，分别表示树的前序遍历和中序遍历。字符仅包括大小写英文字母和数字，最多 62 个，且保证一棵二叉树内不存在相同的节点。如果给出的前序遍历和

中序遍历对应存在一棵二叉树，则输出其后序遍历；否则输出 "Error"。

2.4.2 题目要求

输入格式：

每行包含两个字符串，分别表示前序遍历和中序遍历。

输出格式：

对应每一行输入，输出一行后序遍历结果；如果无法构建出对应的二叉树，则输出 "Error"。

2.4.3 数据结构设计

```
DEFINE OK AS 0
DEFINE ERROR AS -1
DEFINE SOVERFLOW AS -1

DEFINE ElemType AS char
DEFINE Status AS int
```

2.4.4 功能实现

```
FUNCTION print(IN preOrder AS string, IN middleOrder AS string, OUT result AS vector of ElemType) RETURNS Status
    IF preOrder IS EMPTY OR middleOrder IS EMPTY THEN
        RETURN OK

    root = preOrder[0]
    pos = FIND middleOrder FOR root

    IF pos IS NOT FOUND THEN
        RETURN ERROR

    ret1 = CALL print(preOrder[1:pos], middleOrder[0:pos], result)
    ret2 = CALL print(preOrder[pos + 1:], middleOrder[pos + 1:], result)

    IF ret1 == ERROR OR ret2 == ERROR THEN
        RETURN ERROR
    ELSE
        APPEND root TO result

    RETURN OK
END FUNCTION
```

```

FUNCTION main()
    DECLARE preOrder AS string
    DECLARE middleOrder AS string

    WHILE READ preOrder AND middleOrder DO
        DECLARE result AS vector of ElemType

        ret = CALL print(preOrder, middleOrder, result)
        IF ret == ERROR THEN
            PRINT "Error"
        ELSE
            FOR EACH ch IN result DO
                PRINT ch
            END FOR
        END IF

        PRINT newline
    END WHILE
END FUNCTION

```

数据结构：

ElemType: 定义为字符类型，用于存储树节点的数据。

Status: 定义为整型，用于表示函数的返回状态（如成功、错误等）。

result: 用于存储后序遍历的结果。

功能实现：

print 函数: 根据前序和中序遍历构建后序遍历，递归实现。

main 函数: 读取输入的前序和中序遍历，调用 **print** 函数并输出结果。

2.4.5 遇到的问题

树的构建: 在根据前序和中序遍历构建树时，如何正确地划分左右子树的范围是一个挑战。

解决方法: 通过查找根节点在中序遍历中的位置，确定左右子树的节点范围，并递归构建。

2.4.6 心得体会

在实现这个问题的过程中，我加深了对二叉树遍历的理解，特别是前序遍历和中序遍历之间的关系。通过递归构建树的过程，我学会了如何利用前序遍历的第一个元素确定根节点，并通过中序遍历找到左右子树的范围。这种方法不仅提高了我的逻辑思维能力，还让我更加熟悉了递归的应用。

此外，处理错误情况（如无法构建树的情况）让我意识到在编程中考虑边界条件的重要性。通过这次实践，我对树的构建和遍历有了更深入的理解，并为今后的编程学习打下了坚实的基础。

2.5 题目五

树的重构

2.5.1 题目描述

在计算机科学中，树是一种重要的数据结构，其中有序树是一种特殊类型的树，允许每个节点有可变数量的子节点。为了更方便地处理有序树，通常将其转换为二叉树。转换的过程包

括去除每个节点与其子节点的边，并将第一个孩子节点作为左孩子，后续兄弟节点作为右孩子。这个转换可能会导致树的深度增加，因此需要计算转换前后的树的深度。

2.5.2 题目要求

输入格式：

输入由多行组成，每一行表示一棵树的深度优先遍历方向，使用字符 **d** 表示下行（进入子节点），使用字符 **u** 表示上行（返回父节点）。

输入以 **#** 开头的行结束。

输出格式：

对于每棵树，输出格式为 **Tree t: h1 => h2**，其中 **t** 是样例编号（从 1 开始），**h1** 是转换前的树的深度，**h2** 是转换后的树的深度。

2.5.3 数据结构设计

```
DEFINE OK AS 0
DEFINE ERROR AS -1
DEFINE SOVERFLOW AS -1
DEFINE LIST_INIT_SIZE AS 100
DEFINE LIST_INCRESEMENT AS 10
DEFINE START_WITH_ZERO AS 1

DEFINE ElemType AS int
DEFINE Status AS int

STRUCT Tree_Node
    ElemType data           // 节点数据
    Tree_Node* ltree        // 左子树指针
    Tree_Node* rtree        // 右子树指针
END STRUCT

DEFINE Bitree AS POINTER TO Tree_Node
```

Tree_Node：定义了树节点的结构，包含节点数据和指向左右子树的指针。

Bitree：定义为指向 **Tree_Node** 的指针，用于表示树的根节点。

2.5.4 功能实现

```
FUNCTION CreateBiTree(OUT T AS Bitree, IN input AS string) RETURNS Status
STRUCT VisitNode
    Bitree node
    bool visit
    CONSTRUCTOR VisitNode(IN nn AS Bitree = NULL, IN vv AS bool = false)
        node = nn
        visit = vv
    END CONSTRUCTOR
END STRUCT

DECLARE q AS VisitNode
DECLARE stack AS stack of VisitNode
DECLARE last AS Bitree = NULL
DECLARE p AS Bitree = NEW Tree_Node

IF p IS NULL THEN
    EXIT(SOVERFLOW)

p->ltree = NULL
p->rtree = NULL
```



```

stack.push(VisitNode(p, false))
T = p

FOR EACH ch IN input DO
    SWITCH ch
        CASE 'd':
            q = stack.top()
            stack.pop()
            p = NEW Tree_Node
            IF p IS NULL THEN
                EXIT(SOVERFLOW)

            p->ltree = NULL
            p->rtree = NULL

            IF NOT q.visit THEN
                q.visit = true
                q.node->ltree = p
                stack.push(q)
            ELSE

```

```

                stack.push(q)
                last->rtree = p

                stack.push(VisitNode(p, false))
                BREAK
            CASE 'u':
                q = stack.top()
                stack.pop()
                last = q.node
                BREAK
        END SWITCH
    END FOR

    RETURN OK
END FUNCTION

FUNCTION Getdepth(IN T AS Bitree) RETURNS Status
    IF T IS NULL THEN
        RETURN 0

```

```

    ldepth = Getdepth(T->ltree)
    rdepth = Getdepth(T->rtree)

    RETURN MAX(ldepth, rdepth) + 1
END FUNCTION

FUNCTION Get_input_depth(IN order AS string) RETURNS Status
    DECLARE depth AS int = 0
    DECLARE max_depth AS int = 0

    FOR EACH ch IN order DO
        SWITCH ch
            CASE 'd':
                depth++
                max_depth = MAX(max_depth, depth)
                BREAK
            CASE 'u':
                depth--
                BREAK
        END SWITCH
    END FOR

    IF depth < 0 THEN
        RETURN ERROR
    END IF

    RETURN max_depth
END FUNCTION

```

```

    IF depth < 0 THEN
        RETURN ERROR
    END IF

    RETURN max_depth
END FUNCTION

FUNCTION main()
    DECLARE root AS Bitree
    DECLARE order AS string
    DECLARE n_ary_depth AS int = 0
    DECLARE depth AS int = 0

    FOR i FROM 1 TO INFINITY DO
        READ order
        IF order == "#" THEN
            BREAK
        END IF
    END FOR

```

```

n_ary_depth = Get_input_depth(order)
CreateBiTree(root, order)
depth = Getdepth(root)
depth = (START_WITH_ZERO) ? depth - 1 : depth

PRINT "Tree " + i + ": " + n_ary_depth + " => " + depth
END FOR
END FUNCTION

```

CreateBiTree: 根据输入的深度优先遍历构建二叉树。

Getdepth: 计算二叉树的深度。

Get_input_depth: 计算输入序列的深度。

main: 主函数，读取输入，构建树并输出多叉树和二叉树的深度。

2.5.5 遇到的困难

树的深度计算：在处理输入时，如何准确地计算树的深度是一个挑战。

解决方法：我使用了一个计数器来跟踪当前的深度，并在遇到 **d** 时增加深度，遇到 **u** 时减少深度。

转换后的深度计算：在转换为二叉树后，如何正确计算新的深度也是一个难点。

解决方法：我分析了转换规则，发现每个节点的第一个孩子成为左孩子，后续兄弟成为右孩子，因此可以通过遍历原树的结构来计算新的深度。

输入处理：处理多行输入并在遇到结束符时停止是一个常见问题。

解决方法：我使用了循环读取输入，直到遇到以 **#** 开头的行，从而有效地管理输入数据。

2.5.6 心得体会

在实现这个问题的过程中，我加深了对树结构的理解，特别是如何通过深度优先遍历来计算树的深度。通过分析输入的 **d** 和 **u** 字符，我能够有效地模拟树的结构，并计算出转换前后的深度。这种方法让我更加熟悉了栈的使用，因为在处理深度优先遍历时，栈是一个非常有效的工具。

此外，处理树的深度变化让我意识到在设计数据结构时，深度和复杂度的关系是非常重要的。通过这次实践，我对树的操作有了更深入的理解，并为今后的编程学习打下了坚实的基础。

2.6 题目五

寻找最近祖先

2.6.1 题目描述

给定一颗多叉树，要求找出两个节点的最近公共祖先（LCA）。一个节点的祖先节点可以是该节点本身，树中任意两个节点都至少有一个共同祖先，即根节点。

2.6.2 题目要求

输入的第一行是测试样本数 **T**。

每个测试样本的第一行包含两个整数 **Ni**（节点数）和 **Mi**（询问数）。

接下来 **Ni-1** 行，每行包含两个整数 **a** 和 **b**，表示 **a** 是 **b** 的父节点。

接下来 M 行，每行包含两个整数 x 和 y ，表示询问 x 和 y 的共同祖先。

2.6.3 数据结构

```
DEFINE FALSE AS -1
DEFINE OK AS 0
DEFINE ERROR AS -1
DEFINE SOVERFLOW AS -1
DEFINE NO_PARENT AS -1 // 没有双亲结点

DEFINE ElemType AS int
DEFINE Status AS int

STRUCT Tree_Node
    ElemType data // 节点数据
    Tree_Node* parent // 父节点指针
    CONSTRUCTOR Tree_Node()
        data = NO_PARENT
        parent = NULL
    END CONSTRUCTOR
END STRUCT

DEFINE Bitree AS POINTER TO Tree_Node

// 全局变量
Bitree myinfo // 存储树的结构
int* depth // 存储每个节点的深度
```

2.6.4 功能实现

```

FUNCTION Create_Tree(IN Nodes AS int) RETURNS Status
    DECLARE parent AS ElemType
    DECLARE son AS ElemType

    IF Nodes <= 0 THEN
        RETURN ERROR

    FOR i FROM 1 TO Nodes - 1 DO
        READ parent, son
        myinfo[son - 1].parent = &myinfo[parent - 1] // 设置父节点
        myinfo[son - 1].data = son // 记录子节点数据
        myinfo[parent - 1].data = parent // 记录父节点数据
    END FOR

    RETURN OK
END FUNCTION

FUNCTION Create_depth_info(IN Nodes AS int) RETURNS Status
    FOR i FROM 0 TO Nodes - 1 DO
        depth[i] = 0 // 初始化深度为0

        DECLARE current AS Bitree = &(myinfo[i]) // 当前节点

        WHILE current->parent != NULL DO
            depth[i]++ // 深度加一
            current = current->parent // 移动到父节点
        END WHILE
    END FOR

    RETURN OK
END FUNCTION

FUNCTION Find_Ancessor(IN p AS Bitree, IN q AS Bitree) RETURNS Bitree
    DECLARE p_data AS ElemType = p->data
    DECLARE q_data AS ElemType = q->data

    WHILE depth[p_data - 1] > depth[q_data - 1] DO
        p = p->parent // p上移
        p_data = p->data
    END WHILE

```



```

    WHILE depth[p_data - 1] < depth[q_data - 1] DO
        q = q->parent // q上移
        q_data = q->data
    END WHILE

    WHILE p->data != q->data DO
        p = p->parent
        q = q->parent

        IF p == NULL OR q == NULL THEN
            BREAK // 处理没有共同祖先的情况
        END IF
    END WHILE

    RETURN p // 返回共同祖先
END FUNCTION

FUNCTION main() RETURNS int
    DECLARE Nodes AS int
    DECLARE problems AS int

```

```

    DECLARE num_samples AS int
    READ num_samples

    FOR i FROM 0 TO num_samples - 1 DO
        READ Nodes, problems

        myinfo = NEW Tree_Node[Nodes] // 申请树的存储空间
        IF myinfo IS NULL THEN
            EXIT(SOVERFLOW)

        depth = NEW int[Nodes] // 申请深度存储空间
        IF depth IS NULL THEN
            EXIT(SOVERFLOW)

        Create_Tree(Nodes) // 构建树
        Create_depth_info(Nodes) // 计算深度

        FOR j FROM 0 TO problems - 1 DO
            DECLARE p_data AS ElemType
            DECLARE q_data AS ElemType

```

```

        READ p_data, q_data
        DECLARE p AS Bitree = &myinfo[p_data - 1] // 获取节点p
        DECLARE q AS Bitree = &myinfo[q_data - 1] // 获取节点q

        DECLARE ancestor AS Bitree = Find_Anccestor(p, q) // 查找共同祖先

        IF ancestor IS NOT NULL THEN
            PRINT ancestor->data // 输出共同祖先
        ELSE
            PRINT "No common ancestor" // 处理没有共同祖先的情况
        END IF
    END FOR

    DELETE[] myinfo // 释放树的存储空间
    DELETE[] depth // 释放深度存储空间
END FOR

RETURN 0
END FUNCTION

```

2.6.5 遇到的困难

树的构建：在处理多叉树时，如何有效地构建树结构并存储父子关系是一个挑战。

解决方法：使用邻接表或字典来存储每个节点的子节点，并维护一个父节点的映射。

最近公共祖先的查找：如何高效地找到两个节点的最近公共祖先，尤其是在多叉树中。

解决方法：使用深度优先搜索（DFS）记录每个节点的深度和父节点信息，利用这些信息来查找 LCA。

多次查询的效率：在多次查询的情况下，如何优化 LCA 的查找过程。

解决方法：使用预处理方法，如 Tarjan 算法或倍增法，来加速 LCA 查询。

2.6.6 心得体会

在实现最近公共祖先的过程中，我加深了对树结构的理解，特别是多叉树的构建和遍历。通过使用 DFS，我能够有效地记录每个节点的深度和父节点信息，从而快速找到 LCA。这次实践让我认识到数据结构的选择对算法效率的重要性，尤其是在处理大规模数据时。

此外，处理多次查询时的优化让我意识到预处理的重要性。通过合理的算法设计，可以显著提高查询效率，减少不必要的计算。这次学习让我对树的操作有了更深入的理解，并为今后的编程学习打下了坚实的基础。

2.7 题目

表达式树

2.7.1 题目描述

在这道题目中，我们需要将一个中缀表达式转换为表达式树，并输出其后缀表达式（逆波兰式）、树的结构以及计算该表达式的值。中缀表达式可能包含小写字母（变量）和运算符（+、-、*、/、小括号），并且输入的变量会有对应的值。

2.7.2 题目要求

输入格式：

第一行是中缀表达式（长度不超过 50）。

第二行是一个整数 n ($n \leq 10$)，表示中缀表达式中的变量数。

接下来的 n 行，每行包含一个变量及其对应的值。

输出格式：

第一部分：该表达式的逆波兰式。

第二部分：表达式树的结构。

第三部分：将值代入变量后，该中缀表达式的计算结果。

2.7.3 数据结构

```
// 定义元素类型
ElemType = char
Status = int

// 定义树节点结构
Tree_Node:
    data: ElemType           // 节点存储的数据（运算符或变量名）
    ltree: Tree_Node*        // 指向左子树的指针
    rtree: Tree_Node*        // 指向右子树的指针

// 定义代数结构体
Algebra:
    name: ElemType           // 变量名
    value: int               // 变量的值
```

ree_Node：表示表达式树的节点，包含数据和左右子树的指针。

Algebra：表示变量及其对应的值。

2.7.4 功能实现

```
// 创建子树的函数
Create_son_Tree(value_stack, operator_stack):
    a = value_stack.pop() // 弹出右子树
    b = value_stack.pop() // 弹出左子树
    op = operator_stack.pop() // 弹出运算符
    op.ltree = b           // 将左子树连接到运算符节点
    op.rtree = a           // 将右子树连接到运算符节点
    value_stack.push(op)   // 将新创建的子树推入值栈

// 构造表达式树的函数
Create_Tree(root):
    value_stack = new stack<Bitree>() // 值栈
    operator_stack = new stack<Bitree>() // 运算符栈
    for each character ch in input_expression:
        if ch is an operator or '(' or ')':
            // 根据运算符的优先级处理栈
            // 省略具体的优先级处理逻辑
        else:
            // ch 是变量，创建新节点并推入值栈
            value_stack.push(new Tree_Node(ch))
```

```

// 清空运算符栈, 构建完整的树
while operator_stack is not empty:
    Create_son_Tree(value_stack, operator_stack)
root = value_stack.top() // 获取树的根节点

// 后序遍历输出的函数
Post_Order(root, visit_function):
    if root is null:
        return OK
    Post_Order(root.ltree, visit_function)
    Post_Order(root.rtree, visit_function)
    visit_function(root.data)

// 计算树的深度的函数
GetTreeDepth(root):
    if root is null:
        return 0
    return 1 + max(GetTreeDepth(root.ltree), GetTreeDepth(root.rtree))

// 打印二叉树的函数

// 打印二叉树的函数
printTree(root):
    depth = GetTreeDepth(root)
    // 创建一个二维数组或向量来存储树的字符串表示
    // 使用队列进行层次遍历, 填充树的结构
    // 输出树的结构

// 计算表达式值的函数
Cal_Tree(root, algebra_array, n):
    if root is null:
        return 0
    if root.ltree is null and root.rtree is null: // 叶子节点
        for each algebra in algebra_array:
            if algebra.name == root.data:
                return algebra.value
    ltree_value = Cal_Tree(root.ltree, algebra_array, n)
    rtree_value = Cal_Tree(root.rtree, algebra_array, n)
    // 根据运算符计算值并返回

```

Create_son_Tree: 用于创建子树并将其推入值栈。

Create_Tree: 构建表达式树的主要函数, 处理输入的中缀表达式。

Post_Order: 后序遍历树并执行访问操作。

GetTreeDepth: 计算树的深度。

printTree: 打印树的结构。

Cal_Tree: 计算表达式树的值。

这个伪代码提供了数据结构和主要功能的概述, 便于理解代码的逻辑和结构。

2.7.5 遇到的困难

表达式树的构建：

需要正确处理运算符的优先级和括号的嵌套，确保树的结构符合运算顺序。

需要设计合适的数据结构来表示树节点。

逆波兰式的生成：

需要实现一个算法（如 Shunting Yard 算法）来将中缀表达式转换为后缀表达式。

树的可视化：

需要根据树的高度和节点的排列规则，计算每个节点的输出位置，确保树的结构清晰可读。

计算表达式的值：

需要实现一个计算函数，能够根据树的结构和节点的值进行递归计算。

2.7.6 心得体会

数据结构的重要性：在处理表达式树时，选择合适的数据结构（如链表、栈、树）是非常关键的。树的节点需要包含运算符和子节点的引用，以便于后续的遍历和计算。

算法的实现：实现中缀到后缀的转换时，理解运算符优先级和括号的处理是关键。通过使用栈来管理运算符，可以有效地实现这一过程。

可视化的挑战：将树结构以特定格式输出需要仔细计算每个节点的位置，尤其是在处理不满的二叉树时，如何填充空格以保持格式的整齐是一个挑战。

调试与测试：在实现过程中，频繁的测试和调试是必要的，尤其是在处理边界情况（如只有一个变量或复杂的括号嵌套）时，确保程序的健壮性。

3.实验总结

表达式树是一种树形结构，用于表示数学表达式，其中叶子节点表示操作数，非叶子节点表示运算符。最近公共祖先是指在二叉树中两个节点的最近共同祖先。树的重构是根据遍历结果重建树的过程。后序遍历是获取树节点的后序访问顺序的方法。感染二叉树需要的总时间是指从某个节点开始传播到整个树所需的时间。二叉树的同构是指两棵二叉树在结构上相同但节点值可以不同。二叉树的非递归遍历是指使用栈等数据结构实现的遍历方式。